

S a a S P r o d u c t D e v e l o p m e n t P l a n

Madrasa Management System

Complete Product Blueprint & Development Plan

Multi-Tenant SaaS Platform for Islamic Educational Institutions

Version 2.0 | Comprehensive Edition | Confidential

Prepared by: Development Team

Date: June 2026

Table of Contents

1. Executive Summary

The Madrasa Management System (MMS) is a comprehensive, multi-tenant SaaS platform designed specifically for Islamic educational institutions (Qawmi, Alia, Hifz Madrasa, and Nizami systems). Unlike generic school management systems, MMS is built from the ground up to address the unique academic, spiritual, and administrative requirements of madrasa education. This includes specialized modules for Quran memorization (Hifz) tracking with three-tier revision management (Sabak, Sabaq Para, Dhor), Islamic worship tracking (Muhasaba), student financial welfare through Zakat/Sadaqah disbursement automation, Tajweed assessment, and comprehensive Islamic curriculum management.

The platform operates on a Software-as-a-Service model, allowing individual madrasas to subscribe, configure their institution, and manage all operations through a modern web interface. A multi-tenant architecture ensures complete data isolation between institutions, while shared infrastructure keeps operational costs low. The system supports three languages (Bangla, English, Arabic with RTL) and integrates with Bangladesh-specific payment gateways (bKash, Nagad, SSL Commerz) and SMS providers for communication.

This blueprint document serves as the master development plan, covering system architecture, the complete database schema with 40+ tables, world-class UI/UX design specifications for every module, security policies, deployment strategy, and a phased development roadmap. Every feature described in this document has been designed with the specific workflows of madrasa administrators, teachers, students, and guardians in mind.

- 3-Tier Hifz Tracking: Sabak (new memorization), Sabaq Para (current para revision), Dhor (old para revision) with automated scheduling and progress visualization.
- Muhasaba Module: Daily self-accounting of Islamic worship practices including Salah, Quran recitation, Dhikr, fasting, and optional Sunnah acts with streak tracking and teacher oversight.
- Student Wallet System: Automated Zakat/Sadaqah fund disbursement with tamlik (distribution) tracking, transaction history, and need-based allocation.
- Tajweed Assessment: Structured evaluation of Quran recitation rules with rubric-based scoring by qualified teachers.
- Islamic Curriculum Streams: Support for multiple madrasa education streams (Qawmi, Alia, Hifz, Nizami) with configurable subject structures.

- **Prayer Time Integration:** Automatic prayer time display based on geographic location, with Salah tracking for students.
- **Arabic/RTL Support:** Full right-to-left layout support for Arabic language throughout the interface.

2. System Overview

The MMS platform is built on a modern technology stack optimized for performance, maintainability, and developer productivity. The frontend uses Next.js 15 with the App Router pattern, providing server-side rendering, automatic code splitting, and excellent SEO capabilities for the public-facing website module. The backend leverages PostgreSQL as the primary database through Prisma ORM, with Row-Level Security (RLS) policies for tenant data isolation.

2.1 Technology Stack

Layer	Technology	Purpose
Frontend Framework	Next.js 15 (App Router)	Server-rendered React with App Router
Language	TypeScript 5.x	Type-safe development
UI Components	shadcn/ui + Radix UI	Accessible, customizable component library
Styling	Tailwind CSS 4	Utility-first CSS framework
Database	PostgreSQL 16	Relational database with RLS support
ORM	Prisma 5.x	Type-safe database access
Authentication	NextAuth.js v5	Session management with JWT
Caching	Redis	Query caching, session store, rate limiting
File Storage	Local + S3-compatible	Uploads, PDFs, backups
SMS	Pluggable (BulkSMS, Twilio, custom)	Multi-provider SMS abstraction
Payments	bKash, Nagad, SSL Commerz	Bangladesh-specific payment gateways
PDF Generation	Puppeteer / @react-pdf/renderer	Report cards, receipts, certificates
Excel Export	ExcelJS	Data export to .xlsx format
Deployment	Docker + Coolify on Contabo VPS	Self-hosted container management

2.2 Multi-Tenancy Architecture

Multi-tenancy is implemented using a shared database with `tenant_id` column isolation pattern. Every table in the database includes a `tenant_id` foreign key that links records to their owning institution. Prisma middleware automatically injects `tenant_id` filters into all queries at runtime, ensuring that no institution can access another institution's data, even if a developer accidentally omits the filter. As a secondary defense layer, PostgreSQL Row-Level Security (RLS) policies enforce tenant isolation at the database engine level.

Each institution is identified by a unique slug (e.g., `darul-uloom-dhaka`) that serves as their subdomain for both the admin dashboard and the public website. The system middleware extracts the tenant from the request and sets the tenant context for the entire request lifecycle. This architecture allows sharing a single database instance and application server across all tenants while maintaining complete data isolation.

3. Modules & Features

The MMS platform consists of 15 interconnected modules that cover every aspect of madrasa administration, from student enrollment to financial management, academic tracking to spiritual development. Each module is designed as a self-contained feature area with its own dashboard, data tables, forms, reports, and API endpoints. The modules share a common design language and navigation structure to ensure consistency across the platform.

Module	Description	Key Features
1. Super Admin	Platform-wide administration	Tenant management, subscription plans, usage analytics, system health
2. Institution Settings	Institution-specific configuration	Branding, academic year, grading scale, custom fields, payment config
3. RBAC & Permissions	Role-based access control	Module/action-level permissions, role templates, audit trail
4. Student Management	Complete student lifecycle	Admission, profile, class assignment, medical info, academic history
5. Teacher Management	Staff and teacher profiles	Teaching load, qualifications, salary, leave management
6. Attendance	Daily attendance tracking	Multi-period, QR code, SMS alerts, monthly reports

Module	Description	Key Features
7. Hifz Tracking	Quran memorization management	3-tier revision (Sabak/Sabaq/Dhor), progress visualization, Tajweed
8. Exam & Results	Examination management	Multi-type exams, mark entry, tabulation, report cards, GPA calc
9. Muhasaba (Ibadah)	Spiritual worship tracking	Salah, Quran recitation, Dhikr, fasting, streak tracking
10. Finance & Wallet	Financial management	Fee structures, student wallet, Zakat/Sadaqah tamlik, expenses
11. SMS & Notifications	Communication	Bulk SMS, templates, delivery tracking, in-app notifications
12. Website / CMS	Public website builder	Pages, media, templates, SEO, event calendar, gallery
13. Library	Book management	Catalog, issue/return, overdue fines, multi-language books
14. Hostel	Hostel management	Room/bed allocation, fee tracking, visitor log, complaints
15. Transport	Vehicle management	Fleet tracking, route management, pickup/dropoff, GPS

4. World-Class UI/UX Design System

The UI/UX design system ensures a consistent, professional, and deeply intuitive user experience across every dashboard and module in the MMS platform. The design philosophy follows three core principles: (1) Information Density with Clarity, where complex data is presented through structured layouts that prioritize what matters most; (2) Islamic Visual Identity, where the green-white-gold color palette, Arabic typography, and subtle geometric patterns create a distinctively Islamic aesthetic; and (3) Progressive Disclosure, where advanced features are accessible but never overwhelming, allowing new users to be productive immediately while power users can access deeper functionality.

The system targets users with varying levels of technical literacy. Madrasa administrators may range from digitally native youth to older scholars with limited computer experience. Therefore, every interface must pass the grandmother test: can a first-time user complete the most common task (e.g., marking attendance) without training or documentation?

4.1 Design Foundation

Element	Specification	Implementation
Primary Color	Islamic Green (#0F766E / teal-700)	Tailwind: bg-teal-700, used for nav, primary buttons, active states
Secondary Color	Warm Gold (#D4A853)	Tailwind: text-amber-500, used for accents, highlights, badges
Background	Off-White (#F8FAFC)	Tailwind: bg-slate-50, reduces eye strain for long data sessions
Surface Color	White (#FFFFFF)	Cards, modals, forms; with subtle shadows for elevation hierarchy
Text Primary	Dark Slate (#1E293B)	Tailwind: text-slate-800, high contrast for readability
Text Secondary	Medium Gray (#64748B)	Tailwind: text-slate-500, labels, metadata, timestamps
Error/Warning	Red (#EF4444) / Amber (#F59E0B)	Validation errors, overdue alerts, critical notifications
Success	Green (#22C55E)	Confirmations, completed actions, positive balances
Font (Latin)	Inter (Google Fonts)	Clean, modern sans-serif optimized for UI
Font (Bangla)	Noto Sans Bengali	Full Unicode Bangla support with proper ligatures
Font (Arabic)	Noto Naskh Arabic / Amiri	Traditional Naskh script for Quranic text display
Border Radius	8px (rounded-lg)	Cards, buttons; 12px (rounded-xl) for modals and large cards
Spacing Unit	4px grid (Tailwind scale)	Consistent spacing using Tailwind spacing scale (p-4, m-6, gap-3)
Shadow	shadow-sm / shadow-md / shadow-lg	Three-level elevation hierarchy for cards, dropdowns, modals
Animation	150ms ease transitions	Subtle hover/active states, no distracting animations

4.2 Layout Architecture

The application uses a persistent sidebar + topbar layout that maximizes screen real estate for data display. The sidebar collapses to icons on smaller screens and expands on hover. Every module page follows the same structural pattern: Breadcrumb Navigation (contextual path), Page Title + Action Bar (title on left, primary action buttons on right),

Filter/Search Bar (collapsible), Data Table or Card Grid (main content), and Pagination/Footer (bottom controls). This consistent structure means that once a user learns one module, they can navigate any other module without relearning the interface.

4.3 Dashboard-Specific UI Guidelines

Each dashboard in the MMS platform is designed with a specific information hierarchy that matches the module's primary use case. The following guidelines ensure that every dashboard delivers the best possible user experience:

- **Layout:** Full-width metric cards at top (4 columns), followed by a two-column layout with tenant growth chart (left) and recent activity log (right).
- **Key Widgets:** Total Tenants, Active Subscriptions, Monthly Revenue, System Health Status. Each card uses sparkline mini-charts for trend visualization.
- **Color Coding:** Green for healthy metrics, amber for warnings (e.g., storage approaching limit), red for critical issues (e.g., database backup failure).
- **Interaction:** Click any metric card to drill down into a filtered detail view. Tenant table supports bulk actions (activate/suspend/delete) with checkbox selection.
- **Layout:** Welcome banner with institution name and Islamic greeting, followed by 6 KPI cards in a responsive grid. Below: Two-column layout with attendance summary chart (left) and fee collection overview (right). Bottom: Recent notices, upcoming events, and prayer times sidebar.
- **KPI Cards:** Total Students, Present Today (%), Fee Due This Month, Pending Tasks, Upcoming Events, Hifz Progress (for institutions with Hifz program). Each card shows trend arrow (up/down) compared to previous month.
- **Quick Actions Bar:** A floating action bar at the bottom with the 5 most common actions (Mark Attendance, Add Student, Send SMS, Record Payment, Create Notice) as icon buttons for one-tap access.
- **Prayer Time Widget:** Displays current prayer time, next prayer countdown, and Iqamah time based on institution location. Uses a subtle card with Islamic green accent border.
- **Data Table:** Full-featured table with sortable columns, multi-column filtering (class, section, status, gender), bulk selection, and row actions (view, edit, print ID card).
- **Student Card View:** Toggle to card grid view showing student photo, name, class, roll number, and quick stats. Useful for visual identification by class teachers.

- Admission Form: Multi-step wizard (Personal Info > Guardian Info > Academic Info > Document Upload > Review & Submit) with progress indicator and auto-save draft.
- Profile Page: Tabbed layout with sections for Basic Info, Academic History, Attendance Chart, Fee Status, Hifz Progress (if applicable), Muhasaba Summary, and Document Gallery. Each tab loads independently for performance.
- Main View: A visual grid/table showing all Hifz students with color-coded status for each Juz (Memorized = green, Revision Needed = amber, Not Started = gray). This provides an at-a-glance view of each student's overall Hifz progress.
- Student Detail Page: Three-tab layout for Sabak (new lesson), Sabaq Para (current para review), and Dhor (old para revision). Each tab shows the specific ayah range, revision schedule, last revision date, and teacher remarks.
- Progress Visualization: A progress bar showing total Juz completed out of 30, with individual Surah-level breakdown. Streak indicator showing consecutive days of revision.
- Teacher Interface: Quick entry panel for daily revision status (Present/Absent/Weak/Strong) with bulk entry mode for entire class at once. Tajweed assessment quick-rating panel.
- Calendar View: Full monthly calendar showing daily ibadah completion as colored dots (green = all 5 prayers, amber = partial, red = missed). Click any day to see detailed breakdown.
- Daily Entry Form: Simple checkbox-based form for 5 Fard Salah, Sunnah prayers, Quran recitation (pages/surah), Dhikr count, fasting status, and optional notes. One-tap submission with confirmation toast.
- Streak System: Visual streak counter with flame icon, consecutive day count, and personal best record. Gamified badges for milestones (7-day streak, 30-day streak, 100-day streak).
- Teacher Oversight: Aggregate view showing class-wide Muhasaba statistics. Identify students who are struggling spiritually and need attention. Anonymous comparison charts (student names hidden) for motivational display.
- Overview Cards: Total Revenue, Pending Fees, Student Wallet Balance (Zakat Fund), Monthly Expense, Net Balance. Each with month-over-month comparison.
- Fee Collection Table: Filterable by class, status (paid/partial/overdue), and payment method. Color-coded rows: green for paid, amber for partial, red for overdue. Bulk SMS action for overdue reminders.

- **Student Wallet Interface:** Individual student wallet page showing Zakat/Sadaqah credits, disbursement history, remaining balance, and need-based priority score. Transaction receipt generation.
- **Zakat Tamlik Panel:** Dedicated panel for managing Zakat fund collection, eligibility assessment of students based on need analysis, automated disbursement suggestions, and audit trail for compliance.
- **Quick Entry Mode:** Grid layout showing all students in a class with radio buttons for Present/Absent/Late/Excused. One-click submit for the entire class. Auto-saves every 30 seconds to prevent data loss.
- **Alternative Entry:** QR code scanning mode where each student has a QR code ID card. Teacher scans with phone camera for instant attendance recording.
- **Analytics View:** Monthly heatmap showing attendance patterns by day-of-week. Trend line showing class attendance rate over time. Individual student attendance percentage with comparison to class average.
- **Exam Setup:** Wizard for creating exams with subject selection, date/time, total marks, and pass marks. Supports multiple exam types (monthly, quarterly, midterm, final).
- **Mark Entry:** Spreadsheet-style mark entry grid with automatic total calculation, grade assignment based on configurable grading scale, and validation for out-of-range marks.
- **Tabulation Sheet:** Auto-generated tabulation view showing all subjects in columns, student-wise marks, totals, grades, and GPA. Export to PDF for board submission.
- **Report Card:** Beautifully formatted PDF report card with institution branding, student photo, subject-wise marks, grades, teacher remarks, and class position. Supports both Bangla and English templates.

4.4 Responsive & Accessibility

- **Breakpoints:** sm(640px), md(768px), lg(1024px), xl(1280px). Sidebar collapses to icon-only on tablet, becomes bottom navigation on mobile.
- **Touch Targets:** Minimum 44x44px tap targets for all interactive elements on mobile. Adequate spacing between buttons to prevent accidental taps.
- **Keyboard Navigation:** Full keyboard support with Tab/Shift+Tab navigation, Enter to activate, Escape to close modals. Focus indicators visible on all interactive elements.

- RTL Support: Automatic layout mirroring when Arabic language is selected. Text alignment, sidebar position, and reading order all adapt to RTL.
- Dark Mode: System-aware dark theme using CSS custom properties. Toggle available in user settings. Respects prefers-color-scheme media query.
- Loading States: Skeleton screens for data tables, progress indicators for form submissions, optimistic UI updates for actions like attendance marking.

5. Database Schema Design

The database schema is the backbone of the MMS platform. It follows a normalized relational design optimized for PostgreSQL, with every table including a `tenant_id` foreign key for multi-tenancy isolation, standard audit columns (`created_at`, `updated_at`), and soft-delete support where appropriate. The schema is managed through Prisma migrations for version-controlled schema evolution.

This section provides the complete database design with all 40+ tables organized by domain. Each table definition includes column names, data types, constraints, relations, and indexes. The design specifically addresses the unique requirements of madrasa education that generic school management systems fail to capture: three-tier Hifz tracking (Sabak, Sabaq Para, Dhor), daily ibadah accounting (Muhasaba), student financial welfare through Zakat/Sadaqah wallet system, Tajweed assessment, Islamic curriculum streams, and prayer time tracking.

5.1 Enumerations & Data Types

Enumerations enforce data integrity by restricting column values to predefined sets. The following enums are used throughout the schema:

Enum Name	Values	Used In
TenantPlan	FREE, BASIC, PRO, ENTERPRISE	Tenant.plan
TenantStatus	ACTIVE, SUSPENDED, TRIAL, CANCELLED	Tenant.status
Gender	MALE, FEMALE, OTHER	Student, User, Guardian
BloodGroup	A_PLUS, A_MINUS, B_PLUS, B_MINUS, AB_PLUS, AB_MINUS, O_PLUS, O_MINUS	Student
MadrasaStream	QAWMI, ALIA, HIFZ, NIZAMI, GENERAL	Class.stream, InstitutionSettings
HifzTier	SABAK, SABAQ_PARA, DHOR	HifzProgress.tier

Enum Name	Values	Used In
HifzStatus	NEW, MEMORIZING, MEMORIZED, REVISION_NEEDED, COMPLETED, STAGNANT	HifzProgress.status
TajweedGrade	EXCELLENT, GOOD, SATISFACTORY, NEEDS_IMPROVEMENT	TajweedAssessment.grade
AttendanceStatus	PRESENT, ABSENT, LATE, LEAVE, EXCUSED	Attendance.status
ExamType	MONTHLY, QUARTERLY, MIDTERM, FINAL, SPECIAL, HIFZ_TEST	Exam.type
FeeStatus	PENDING, PARTIAL, PAID, OVERDUE, WAIVED, REFUNDED	FeeAssignment.paymentStatus
FeeCategory	ADMISSION, TUITION, EXAM, LIBRARY, HOSTEL, TRANSPORT, ZAKAT_FUND, OTHER	FeeAssignment.category
PaymentMethod	CASH, BKASH, NAGAD, SSL_COMMERZ, BANK_TRANSFER, ONLINE	Transaction.method
TransactionType	FEE_PAYMENT, SALARY, EXPENSE, DONATION, ZAKAT_IN, ZAKAT_OUT, SADAQAH_IN, SADAQAH_OUT, REFUND	Transaction.type
WalletTransactionType	ZAKAT_CREDIT, ZAKAT_DEBIT, SADAQAH_CREDIT, SADAQAH_DEBIT, SCHOLARSHIP_CREDIT, EMERGENCY_DEBIT	WalletTransaction.type
NoticePriority	LOW, MEDIUM, HIGH, URGENT	Notice.priority
NoticeStatus	DRAFT, PUBLISHED, ARCHIVED	Notice.status
SMSStatus	PENDING, SENT, DELIVERED, FAILED	SMSLog.status
BookLanguage	ARABIC, BENGALI, ENGLISH, URDU, OTHER	Book.language
IssueStatus	ISSUED, RETURNED, OVERDUE, LOST	BookIssue.status
LeaveStatus	PENDING, APPROVED, REJECTED, CANCELLED	Leave.status

Enum Name	Values	Used In
IbadahType	FAJR, DHUHR, ASR, MAGHRIB, ISHA, SUNNAH, QURAN_RECITATION, DHIKR, FASTING, SADAQAH	MuhasabaEntry.ibadahType
SalahQuality	ON_TIME, LATE, MISSED, WITH_JAMAH, ALONE, QADA	MuhasabaEntry.quality (for Salah)

5.2 Core Infrastructure Tables

These tables form the foundation of the multi-tenant SaaS platform, handling tenant management, authentication, and role-based access control:

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Unique tenant identifier
name	String (200)	NOT NULL	Institution name (English)
nameBn	String (200)	nullable	Institution name (Bangla)
nameAr	String (200)	nullable	Institution name (Arabic)
slug	String (100)	UNIQUE, NOT NULL	URL-safe identifier for subdomain routing
eiin	String (20)	nullable, indexed	Educational Institute Identification Number
logo	String (500)	nullable	Logo file URL/path
address	String	nullable	Full street address
city	String (100)	nullable	City name
district	String (100)	nullable	District/Division
country	String (100)	DEFAULT 'Bangladesh'	Country name
phone	String (20)	nullable	Primary contact phone
email	String	nullable	Primary contact email
website	String	nullable	Institution website URL
plan	TenantPlan	DEFAULT FREE	Current subscription plan
status	TenantStatus	DEFAULT TRIAL	Account status
maxStudents	Int	DEFAULT 100	Student limit based on plan
maxUsers	Int	DEFAULT 10	User limit based on plan

Column	Type	Constraints	Description
latitude	Decimal (10,8)	nullable	Geo coordinates for prayer times
longitude	Decimal (11,8)	nullable	Geo coordinates for prayer times
settings	Json	DEFAULT '{}'	Institution-specific settings (academic year, grading scale, etc.)
branding	Json	DEFAULT '{}'	Theme colors, logo, custom CSS
paymentConfig	Json	DEFAULT '{}'	Payment gateway credentials (encrypted)
smsConfig	Json	DEFAULT '{}'	SMS provider settings
currentAcademicYear	String (20)	nullable	Active academic year identifier
trialEndsAt	DateTime	nullable	Trial period end date
subscription	Subscription?	relation	Linked subscription record
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp
deletedAt	DateTime	nullable	Soft delete timestamp

Indexes: @@unique([slug]), @@index([plan, status])

Relations: users (1:N), students (1:N), classes (1:N), subjects (1:N), exams (1:N), roles (1:N), subscriptions (1:1)

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Subscription identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning tenant
plan	TenantPlan	NOT NULL	Subscribed plan level
status	String	DEFAULT 'ACTIVE'	ACTIVE, EXPIRED, CANCELLED
startsAt	DateTime	NOT NULL	Subscription start date
expiresAt	DateTime	nullable	Subscription expiration date
autoRenew	Boolean	DEFAULT false	Auto-renewal flag

Column	Type	Constraints	Description
transactionId	String	nullable	Last payment transaction reference
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId]), @@index([status, expiresAt])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	User identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
email	String	NOT NULL	Login email address
passwordHash	String	NOT NULL	Bcrypt hashed password
name	String (200)	NOT NULL	Full name (English)
nameBn	String (200)	nullable	Full name (Bangla)
phone	String (20)	nullable	Contact phone number
gender	Gender	nullable	Gender identity
avatar	String (500)	nullable	Profile photo URL
isActive	Boolean	DEFAULT true	Account active status
lastLoginAt	DateTime	nullable	Last successful login timestamp
lastLoginIp	String (45)	nullable	Last login IP address
twoFactorEnabled	Boolean	DEFAULT false	2FA enabled flag
twoFactorSecret	String	nullable	TOTP secret for 2FA
locale	String (5)	DEFAULT 'bn'	Preferred language (bn, en, ar)
theme	String (10)	DEFAULT 'light'	UI theme preference
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp
deletedAt	DateTime	nullable	Soft delete timestamp

Indexes: @@unique([tenantId, email]), @@index([tenantId, isActive])

Relations: tenant (N:1), roles (N:M via UserRole), staffProfile (1:1)

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Role identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Role name (e.g., Admin, Teacher, Accountant)
nameBn	String (100)	nullable	Role name in Bangla
description	String	nullable	Role description
isDefault	Boolean	DEFAULT false	System default role flag
isSuperAdmin	Boolean	DEFAULT false	Platform admin override flag
permissions	Json	DEFAULT '[]'	Module-action permission matrix
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

The permissions JSON structure: [{module: 'students', actions: ['view', 'create', 'edit', 'delete', 'export', 'print', 'approve']}, {module: 'hifz', actions: ['view', 'record', 'assess', 'export']}, ...]

Indexes: @@unique([tenantId, name])

Column	Type	Constraints	Description
userId	String	FK -> User.id, CASCADE	User reference
roleId	String	FK -> Role.id, CASCADE	Role reference
assignedBy	String	FK -> User.id, nullable	Who assigned this role
assignedAt	DateTime	auto now()	Assignment timestamp

Indexes: @@unique([userId, roleId]), @@index([roleId])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Log entry identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
userId	String	FK -> User.id, nullable	User who performed action
action	String (50)	NOT NULL	Action type (CREATE, UPDATE, DELETE, LOGIN, EXPORT, etc.)

Column	Type	Constraints	Description
entity	String (50)	NOT NULL	Entity type (Student, Fee, Exam, etc.)
entityId	String	nullable	Affected record ID
changes	Json	DEFAULT '{}'	Before/after values of changed fields
ipAddress	String (45)	nullable	Client IP address
userAgent	String (500)	nullable	Browser user agent string
createdAt	DateTime	auto now()	Log timestamp

Indexes: @@index([tenantId, entity]), @@index([tenantId, userId]), @@index([createdAt])

5.3 Student & Academic Tables

The student management schema supports the complete student lifecycle from admission to graduation/alumni. It captures personal information, academic enrollment, medical records, guardian details, and links to all student-specific modules (Hifz, Muhasaba, Wallet, Attendance, Fees).

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Student identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
userId	String	FK -> User.id, nullable	Optional linked user account
admissionNo	String (50)	NOT NULL	Institution-generated admission number
rollNo	String (30)	nullable	Class roll number
firstName	String (100)	NOT NULL	First name (English)
firstNameBn	String (100)	NOT NULL	First name (Bangla)
lastName	String (100)	NOT NULL	Last name (English)
lastNameBn	String (100)	nullable	Last name (Bangla)
gender	Gender	NOT NULL	Gender identity
dateOfBirth	DateTime	nullable	Date of birth
bloodGroup	BloodGroup	nullable	Blood group for medical emergencies
photo	String (500)	nullable	Student photo URL

Column	Type	Constraints	Description
classId	String	FK -> Class.id	Current class enrollment
section	String (20)	nullable	Class section (A, B, etc.)
stream	MadrasaStream	nullable	Academic stream (Qawmi, Alia, Hifz, etc.)
admissionDate	DateTime	NOT NULL	Date of enrollment
graduationDate	DateTime	nullable	Date of graduation/completion
status	String	DEFAULT 'ACTIVE'	ACTIVE, INACTIVE, GRADUATED, TRANSFERRED, DROPPED
isHifzStudent	Boolean	DEFAULT false	Enrolled in Hifz program flag
hifzStartDate	DateTime	nullable	Date Hifz program began
hifzTargetDate	DateTime	nullable	Target completion date
previousMadrasa	String (200)	nullable	Previous institution name
transferCertificateNo	String (50)	nullable	TC number from previous school
address	String	nullable	Current residential address
addressBn	String	nullable	Address in Bangla
medicalNotes	String	nullable	Medical conditions/allergies
customFields	Json	DEFAULT '{}'	Institution-defined custom fields
isActive	Boolean	DEFAULT true	Active status flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp
deletedAt	DateTime	nullable	Soft delete timestamp

Indexes: @@unique([tenantId, admissionNo]), @@index([tenantId, classId, isActive]), @@index([tenantId, status]), @@index([tenantId, isHifzStudent])

Relations: tenant (N:1), user (N:1), class (N:1), guardians (1:N), hifzProgress (1:N), muhasabaEntries (1:N), wallet (1:1), attendance (1:N), feeAssignments (1:N), examMarks (1:N)

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Guardian identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Linked student
relationType	String (30)	NOT NULL	FATHER, MOTHER, BROTHER, SISTER, UNCLE, GRANDFATHER, OTHER
name	String (200)	NOT NULL	Guardian name (English)
nameBn	String (200)	nullable	Guardian name (Bangla)
phone	String (20)	NOT NULL	Contact phone (primary)
phone2	String (20)	nullable	Alternative phone number
email	String	nullable	Email address
occupation	String (100)	nullable	Occupation
incomeBracket	String (50)	nullable	Income category for Zakat eligibility
address	String	nullable	Residential address
isPrimary	Boolean	DEFAULT false	Primary contact flag
isEmergencyContact	Boolean	DEFAULT false	Emergency contact flag
photo	String (500)	nullable	Guardian photo (optional)
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, studentId]), @@index([tenantId, phone])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Class identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution

Column	Type	Constraints	Description
name	String (100)	NOT NULL	Class name (e.g., 'Class 1', 'Hifz Level 3')
nameBn	String (100)	nullable	Class name in Bangla
stream	MadrasaStream	nullable	Academic stream
section	String (20)	nullable	Section identifier
classTeacherId	String	FK -> Staff.id, nullable	Assigned class teacher
capacity	Int	nullable	Maximum student capacity
roomNo	String (30)	nullable	Assigned classroom number
academicYear	String (20)	nullable	Academic year this class belongs to
sortOrder	Int	DEFAULT 0	Display order in UI
isActive	Boolean	DEFAULT true	Active class flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, name, section, academicYear]), @@index([tenantId, stream])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Subject identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Subject name (English)
nameBn	String (100)	nullable	Subject name in Bangla
nameAr	String (100)	nullable	Subject name in Arabic
code	String (20)	nullable	Subject code (e.g., QUR-101)
type	String (20)	DEFAULT 'CORE'	CORE, OPTIONAL, ISLAMIC_STUDIES, LANGUAGE, GENERAL
totalMarks	Int	DEFAULT 100	Default total marks
passMarks	Int	DEFAULT 33	Default passing marks
isIslamic	Boolean	DEFAULT false	Islamic studies subject flag

Column	Type	Constraints	Description
description	String	nullable	Subject description
sortOrder	Int	DEFAULT 0	Display order
isActive	Boolean	DEFAULT true	Active flag

Indexes: @@unique([tenantId, code])

Column	Type	Constraints	Description
classId	String	FK -> Class.id, CASCADE	Class reference
subjectId	String	FK -> Subject.id, CASCADE	Subject reference
teacherId	String	FK -> Staff.id, nullable	Assigned teacher
totalMarks	Int	nullable	Override total marks for this class
passMarks	Int	nullable	Override pass marks for this class

Indexes: @@unique([classId, subjectId])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Staff identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
userId	String	FK -> User.id, nullable	Linked user account
employeeId	String (50)	UNIQUE, NOT NULL	Employee ID
firstName	String (100)	NOT NULL	First name
firstNameBn	String (100)	nullable	First name (Bangla)
lastName	String (100)	NOT NULL	Last name
lastNameBn	String (100)	nullable	Last name (Bangla)
gender	Gender	nullable	Gender
phone	String (20)	nullable	Phone number
email	String	nullable	Email
photo	String (500)	nullable	Photo URL
designation	String (100)	NOT NULL	Job title
department	String (100)	nullable	Department
qualification	String	nullable	Educational qualifications
specialization	String (100)	nullable	Subject specialization
isHifzTeacher	Boolean	DEFAULT false	Can assess Hifz students

Column	Type	Constraints	Description
hifzQualification	String (200)	nullable	Hifz/Tajweed certification details
joinDate	DateTime	NOT NULL	Date of joining
salary	Decimal (12,2)	nullable	Monthly salary
bankAccount	String	nullable	Bank account number
address	String	nullable	Residential address
isActive	Boolean	DEFAULT true	Active status
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp
deletedAt	DateTime	nullable	Soft delete timestamp

Indexes: @@unique([tenantId, employeeId]), @@index([tenantId, designation]), @@index([tenantId, isHifzTeacher])

Relations: user (N:1), classes (1:N as classTeacher), subjects (N:M via classSubject), hifzProgress (1:N as assessor), leaves (1:N), payroll (1:N)

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Leave identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
staffId	String	FK -> Staff.id, CASCADE	Staff member
type	String (50)	NOT NULL	SICK, CASUAL, EARNED, MATERNITY, HAJJ, UMRAH, OTHER
startDate	DateTime	NOT NULL	Leave start date
endDate	DateTime	NOT NULL	Leave end date
totalDays	Int	NOT NULL	Total leave days
reason	String	nullable	Leave reason
status	LeaveStatus	DEFAULT PENDING	Approval status
approvedBy	String	FK -> User.id, nullable	Approver reference
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, staffId, status]), @@index([tenantId, startDate])

5.4 Hifz Tracking Tables (3-Tier System)

The Hifz (Quran memorization) tracking system is the most distinctive feature of MMS, designed specifically for the unique requirements of madrasa education. Unlike generic progress tracking, this system implements the traditional three-tier revision methodology that has been used in madrasas for centuries: Sabak (new lesson), Sabaq Para (current para review), and Dhor (old para review). Each tier has its own tracking logic, assessment criteria, and scheduling rules.

This is the master table that tracks each student's memorization status across all 30 Juz of the Quran. Each record represents a specific portion (surah and ayah range) that the student has memorized or is currently working on.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Progress record identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Hifz student reference
juzNumber	Int	NOT NULL	Juz number (1-30)
surahNumber	Int	NOT NULL	Starting Surah number
surahName	String (50)	nullable	Surah name (Arabic)
ayahFrom	Int	NOT NULL	Starting ayah number
ayahTo	Int	NOT NULL	Ending ayah number
tier	HifzTier	NOT NULL	SABAK, SABAQ_PARA, or DHOR
status	HifzStatus	DEFAULT NEW	Current memorization status
memorizedDate	DateTime	nullable	Date this portion was first memorized
lastRevisionDate	DateTime	nullable	Date of last revision
revisionCount	Int	DEFAULT 0	Total revision count
nextRevisionDate	DateTime	nullable	Scheduled next revision date
strengthScore	Int	nullable	1-10 strength rating from teacher
teacherId	String	FK -> Staff.id, nullable	Assessing teacher
teacherRemarks	String	nullable	Teacher feedback/remarks

Column	Type	Constraints	Description
audioRecording	String (500)	nullable	Link to audio recording (optional)
totalAyahs	Int	NOT NULL	Number of ayahs in this portion
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, studentId, juzNumber, surahNumber, ayahFrom]),
 @@index([tenantId, studentId, tier]), @@index([tenantId, studentId, status]),
 @@index([tenantId, nextRevisionDate])

The tier system works as follows:

- **Sabak (New Lesson):** The most recent portion the student is actively memorizing. Reviewed daily. The student reads from memory to the teacher, who marks quality as Strong, Average, or Weak. Weak portions must be repeated the next day.
- **Sabaq Para (Current Para Revision):** The last 5-10 pages that the student has recently memorized beyond Sabak. These are reviewed on alternating days (Sunday-Tuesday-Thursday pattern). The system auto-schedules revisions based on the spaced repetition principle.
- **Dhor (Old Para Revision):** Previously memorized portions that need periodic review to prevent forgetting. These are scheduled based on a rolling schedule (weekly for recent, bi-weekly for older portions). The system tracks revision frequency and alerts when a portion has not been reviewed within the scheduled window.

Automatically generated schedule entries that tell teachers and students which portions to review on which days. The system uses a spaced repetition algorithm that increases revision intervals as the student's mastery strengthens.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Schedule identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
hifzProgressId	String	FK -> HifzProgress.id	Linked progress record

Column	Type	Constraints	Description
tier	HifzTier	NOT NULL	Revision tier
scheduledDate	DateTime	NOT NULL	Scheduled review date
scheduledTime	String (10)	nullable	Scheduled time slot (e.g., '08:00')
status	String	DEFAULT 'PENDING'	PENDING, COMPLETED, MISSED, POSTPONED
completedDate	DateTime	nullable	Actual completion date
qualityRating	Int	nullable	1-5 quality rating
teacherNotes	String	nullable	Teacher notes for this session
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@index([tenantId, studentId, scheduledDate]), @@index([tenantId, scheduledDate, status])

Structured evaluation of Quran recitation quality based on Tajweed rules. Teachers assess students periodically on specific Tajweed rules, providing rubric-based scores that help identify areas for improvement.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Assessment identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student being assessed
teacherId	String	FK -> Staff.id	Assessing teacher
date	DateTime	NOT NULL	Assessment date
juzNumber	Int	nullable	Juz being assessed (optional)
surahFrom	Int	nullable	Starting Surah
ayahFrom	Int	nullable	Starting ayah
overallGrade	TajweedGrade	NOT NULL	Overall Tajweed grade
makharijScore	Int	nullable	1-10 score for articulation points
sifaatScore	Int	nullable	1-10 score for letter characteristics
ghunnahScore	Int	nullable	1-10 score for nasal sounds

Column	Type	Constraints	Description
maddScore	Int	nullable	1-10 score for vowel prolongation
waqfScore	Int	nullable	1-10 score for stopping rules
tizkeerScore	Int	nullable	1-10 score for specific rule areas
totalScore	Int	NOT NULL	Aggregate score (0-100)
comments	String	nullable	Detailed teacher comments
improvementAreas	Json	DEFAULT '[]'	Array of rules needing improvement
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@unique([tenantId, studentId, date]), @@index([tenantId, teacherId, date])

5.5 Muhasaba (Ibadah Tracking) Tables

The Muhasaba module implements the Islamic practice of daily self-accounting (Hisab-e-Nafs). Students record their daily worship activities, and teachers can monitor spiritual development alongside academic progress. The module includes streak tracking, gamified achievements, and anonymous class-wide statistics for motivational purposes.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Entry identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
date	DateTime	NOT NULL	Entry date (no future dates)
ibadahType	IbadahType	NOT NULL	Type of worship (FAJR, DHUHR, ASR, MAGHRIB, ISHA, etc.)
quality	SalahQuality	nullable	Quality rating (for Salah: ON_TIME, LATE, WITH_JAMAH, etc.)
pagesRead	Int	nullable	Quran pages recited (for QURAN_RECITATION type)

Column	Type	Constraints	Description
surahName	String (50)	nullable	Specific Surah name if applicable
dhikrCount	Int	nullable	Number of Dhikr repetitions
isFasting	Boolean	DEFAULT false	Fasting status (for FASTING type)
sadaqahGiven	Boolean	DEFAULT false	Whether Sadaqah was given today
notes	String	nullable	Personal reflection notes
submittedBy	String	FK -> User.id, nullable	Teacher who submitted on behalf (for younger students)
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, studentId, date, ibadahType]), @@index([tenantId, studentId, date]), @@index([tenantId, date, ibadahType])

The Muhasaba system calculates derived metrics: daily completion rate (how many of the 5 daily prayers were performed), weekly average, monthly trend, current streak (consecutive days with all 5 prayers), and personal best streak. These metrics are computed via database views or materialized for dashboard performance.

Pre-computed streak data for quick dashboard access without expensive runtime queries:

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Streak record identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
currentStreak	Int	DEFAULT 0	Current consecutive days with all 5 prayers
longestStreak	Int	DEFAULT 0	Personal best streak
lastStreakDate	DateTime	nullable	Date of last streak contribution
totalCompleteDays	Int	DEFAULT 0	Total days with all 5 prayers (lifetime)

Column	Type	Constraints	Description
monthlyAverage	Decimal (5,2)	DEFAULT 0	Average daily completion for current month
badges	Json	DEFAULT '[]'	Earned achievement badges
updatedAt	DateTime	auto updatedAt	Last update timestamp

Indexes: @@unique([tenantId, studentId])

5.6 Finance & Student Wallet Tables

The financial module handles fee collection, expense management, and a unique Student Wallet system for automated Zakat/Sadaqah disbursement. The wallet system enables transparent management of Islamic charitable funds, ensuring that Zakat collected from donors is distributed (Tamlik) to eligible students based on need-based assessment. Every transaction is fully auditable for Shariah compliance.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Fee structure identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Fee name (e.g., 'Monthly Tuition')
nameBn	String (100)	nullable	Fee name in Bangla
category	FeeCategory	NOT NULL	Fee category
amount	Decimal (12,2)	NOT NULL	Base fee amount
frequency	String (20)	DEFAULT 'MONTHLY'	MONTHLY, QUARTERLY, YEARLY, ONE_TIME
classId	String	FK -> Class.id, nullable	Applicable class (null = all classes)
stream	MadrasaStream	nullable	Applicable stream (null = all streams)
effectiveFrom	DateTime	NOT NULL	Effective start date
effectiveTo	DateTime	nullable	Effective end date
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, category, isActive])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Assignment identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
feeStructureId	String	FK -> FeeStructure.id	Fee structure reference
totalAmount	Decimal (12,2)	NOT NULL	Total fee amount due
paidAmount	Decimal (12,2)	DEFAULT 0	Amount paid so far
waivedAmount	Decimal (12,2)	DEFAULT 0	Amount waived (scholarship/Zakat)
paymentStatus	FeeStatus	DEFAULT PENDING	Current payment status
dueDate	DateTime	nullable	Payment due date
lateFeePerDay	Decimal (8,2)	DEFAULT 0	Late fee charge per day
academicYear	String (20)	nullable	Academic year
remarks	String	nullable	Notes
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, studentId, paymentStatus]), @@index([tenantId, dueDate, paymentStatus])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Payment identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
feeAssignmentId	String	FK -> FeeAssignment.id	Linked fee assignment
studentId	String	FK -> Student.id	Student reference
amount	Decimal (12,2)	NOT NULL	Payment amount
method	PaymentMethod	NOT NULL	Payment method used
transactionRef	String (100)	nullable	Gateway transaction reference
receiptNo	String (50)	nullable	Generated receipt number
paidBy	String	FK -> User.id, nullable	Recorded by
paidByName	String (200)	nullable	Payer name (guardian)
collectedBy	String	FK -> Staff.id, nullable	Staff who collected

Column	Type	Constraints	Description
notes	String	nullable	Payment notes
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@unique([receiptNo]), @@index([tenantId, feeAssignmentId]),
@@index([tenantId, studentId, createdAt])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Expense identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
category	String (50)	NOT NULL	SALARY, UTILITIES, MAINTENANCE, FOOD, TRANSPORT, EVENT, OTHER
amount	Decimal (12,2)	NOT NULL	Expense amount
description	String	nullable	Expense description
date	DateTime	NOT NULL	Expense date
paidTo	String (200)	nullable	Payee name
paidBy	String (FK -> User.id)	nullable	Recorded by
receiptAttachment	String (500)	nullable	Receipt/scanned document URL
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, category, date])

The Student Wallet tracks Zakat/Sadaqah funds allocated to individual students for their financial welfare (Tamlik). This table maintains the running balance and eligibility status for each student.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Wallet identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE, UNIQUE	Linked student
balance	Decimal (12,2)	DEFAULT 0	Current wallet balance
totalCredits	Decimal (12,2)	DEFAULT 0	Lifetime credits received

Column	Type	Constraints	Description
totalDebits	Decimal (12,2)	DEFAULT 0	Lifetime debits/disbursements
isZakatEligible	Boolean	DEFAULT false	Eligible for Zakat disbursement
needScore	Int	nullable	1-100 need assessment score (higher = more needy)
needAssessmentDate	DateTime	nullable	Last need assessment date
needAssessmentNotes	String	nullable	Assessment details
priorityLevel	String (10)	DEFAULT 'NORMAL'	URGENT, HIGH, NORMAL, LOW
isActive	Boolean	DEFAULT true	Active wallet flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, studentId]), @@index([tenantId, isZakatEligible, needScore DESC])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Transaction identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
walletId	String	FK -> StudentWallet.id	Wallet reference
studentId	String	FK -> Student.id	Student reference
type	WalletTransactionType	NOT NULL	ZAKAT_CREDIT, ZAKAT_DEBIT, SADAQAH_CREDIT, etc.
amount	Decimal (12,2)	NOT NULL	Transaction amount (always positive)
balanceBefore	Decimal (12,2)	NOT NULL	Balance before transaction
balanceAfter	Decimal (12,2)	NOT NULL	Balance after transaction
purpose	String (200)	nullable	Purpose of transaction
referenceNo	String (50)	nullable	Reference/receipt number
processedBy	String	FK -> User.id, nullable	Admin who processed

Column	Type	Constraints	Description
donorName	String (200)	nullable	Donor name (for credit transactions)
notes	String	nullable	Additional notes
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@index([tenantId, walletId, createdAt]), @@index([tenantId, studentId, createdAt]), @@index([tenantId, type, createdAt])

5.7 Attendance Tables

The attendance system supports daily tracking for students with multi-period support (morning session, afternoon session), automated SMS alerts for absent students, and comprehensive reporting with monthly/weekly heatmaps.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Attendance record identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id	Student reference
classId	String	FK -> Class.id	Class reference
date	DateTime	NOT NULL	Attendance date
status	AttendanceStatus	NOT NULL	Attendance status
period	String (20)	DEFAULT 'FULL_DAY'	MORNING, AFTERNOON, FULL_DAY, PERIOD_1, etc.
remarks	String	nullable	Remarks (e.g., reason for leave)
recordedBy	String	FK -> User.id, nullable	Teacher who recorded
checkInTime	DateTime	nullable	Check-in timestamp (for QR code mode)
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, studentId, date, period]), @@index([tenantId, classId, date]), @@index([tenantId, date, status])

5.8 Exam & Results Tables

The exam system supports multiple exam types, flexible mark entry with automatic grade/GPA calculation, tabulation sheet generation, and beautiful PDF report card generation with institution branding.

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Exam identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (200)	NOT NULL	Exam name (e.g., 'Final Exam 2026')
nameBn	String (200)	nullable	Exam name in Bangla
type	ExamType	NOT NULL	Exam type
classId	String	FK -> Class.id	Applicable class
startDate	DateTime	NOT NULL	Exam start date
endDate	DateTime	NOT NULL	Exam end date
isPublished	Boolean	DEFAULT false	Results published flag
academicYear	String (20)	nullable	Academic year
remarks	String	nullable	Exam remarks
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, classId, type]), @@index([tenantId, startDate])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Routine identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
examId	String	FK -> Exam.id, CASCADE	Exam reference
subjectId	String	FK -> Subject.id	Subject reference
date	DateTime	NOT NULL	Exam date for this subject
startTime	String (10)	nullable	Start time (HH:MM)
endTime	String (10)	nullable	End time (HH:MM)
totalMarks	Int	NOT NULL	Total marks for this subject
passMarks	Int	NOT NULL	Passing marks

Column	Type	Constraints	Description
roomNo	String (30)	nullable	Exam room assignment

Indexes: @@unique([examId, subjectId]), @@index([tenantId, date])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Mark record identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
examId	String	FK -> Exam.id, CASCADE	Exam reference
studentId	String	FK -> Student.id, CASCADE	Student reference
subjectId	String	FK -> Subject.id, CASCADE	Subject reference
writtenMark	Int	nullable	Written exam marks
oralMark	Int	nullable	Oral/viva marks
practicalMark	Int	nullable	Practical exam marks
totalMark	Int	NOT NULL	Total marks (computed)
grade	String (10)	nullable	Letter grade (A+, A, B, etc.)
gradePoint	Decimal (3,2)	nullable	Grade point (4.00, 3.50, etc.)
isAbsent	Boolean	DEFAULT false	Absent from exam flag
remarks	String	nullable	Remarks
enteredBy	String	FK -> User.id, nullable	Teacher who entered marks
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, examId, studentId, subjectId]), @@index([tenantId, examId, studentId])

5.9 Communication (SMS & Notifications) Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Config identifier
tenantId	String	FK -> Tenant.id, CASCADE, UNIQUE	One config per tenant

Column	Type	Constraints	Description
provider	String (50)	NOT NULL	Provider name (e.g., BULK_SMS, TWILIO, CUSTOM)
apiKey	String (500)	NOT NULL	Encrypted API key
apiUrl	String (500)	nullable	Provider API endpoint
senderId	String (30)	nullable	Sender ID / masked name
isEnabled	Boolean	DEFAULT true	SMS enabled flag
monthlyLimit	Int	DEFAULT 1000	Monthly SMS limit
usedThisMonth	Int	DEFAULT 0	SMS used this month
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Template identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Template name
type	String (50)	NOT NULL	ABSENT_ALERT, FEE_REMINDER, GENERAL, EVENT
content	String (500)	NOT NULL	Template body with placeholders
variables	Json	DEFAULT '[]'	Available placeholder variables
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@index([tenantId, type])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Log identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
recipientPhone	String (20)	NOT NULL	Recipient phone number

Column	Type	Constraints	Description
recipientName	String (200)	nullable	Recipient name
message	String (500)	NOT NULL	Sent message text
status	SMSStatus	DEFAULT PENDING	Delivery status
providerMessageld	String (100)	nullable	Provider message ID for tracking
errorCode	String (20)	nullable	Error code if failed
sentBy	String	FK -> User.id, nullable	User who triggered
sentAt	DateTime	auto now()	Send timestamp
deliveredAt	DateTime	nullable	Delivery confirmation timestamp

Indexes: @@index([tenantId, status]), @@index([tenantId, sentAt])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Notification identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
userId	String	FK -> User.id, CASCADE	Target user
title	String (200)	NOT NULL	Notification title
body	String (500)	NOT NULL	Notification body
type	String (50)	DEFAULT 'INFO'	INFO, WARNING, SUCCESS, ERROR
isRead	Boolean	DEFAULT false	Read status
link	String (500)	nullable	Deep link to relevant page
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@index([tenantId, userId, isRead, createdAt])

5.10 Website / CMS Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Page identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
title	String (200)	NOT NULL	Page title
slug	String (100)	NOT NULL	URL slug

Column	Type	Constraints	Description
content	Text	NOT NULL	Page content (HTML/Markdown)
status	String	DEFAULT 'DRAFT'	DRAFT, PUBLISHED, ARCHIVED
metaTitle	String (200)	nullable	SEO meta title
metaDescription	String (500)	nullable	SEO meta description
sortOrder	Int	DEFAULT 0	Display order
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, slug])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Notice identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
title	String (200)	NOT NULL	Notice title
titleBn	String (200)	nullable	Notice title in Bangla
content	Text	NOT NULL	Notice content (HTML)
priority	NoticePriority	DEFAULT MEDIUM	Priority level
status	NoticeStatus	DEFAULT DRAFT	Publication status
category	String (50)	nullable	Category tag
showOnWebsite	Boolean	DEFAULT false	Display on public website
publishDate	DateTime	nullable	Scheduled publish date
expireDate	DateTime	nullable	Auto-archive date
attachment	String (500)	nullable	Attached file URL
createdBy	String	FK -> User.id	Author
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, status, publishDate])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Event identifier

Column	Type	Constraints	Description
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
title	String (200)	NOT NULL	Event title
titleBn	String (200)	nullable	Title in Bangla
description	Text	nullable	Event description
startDate	DateTime	NOT NULL	Start date/time
endDate	DateTime	nullable	End date/time
venue	String (200)	nullable	Event venue
isPublic	Boolean	DEFAULT true	Show on public website
createdBy	String	FK -> User.id	Organizer
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, startDate])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Media identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
fileName	String (200)	NOT NULL	Original file name
filePath	String (500)	NOT NULL	Stored file path
fileType	String (50)	NOT NULL	MIME type
fileSize	BigInt	NOT NULL	File size in bytes
altText	String (200)	nullable	Alternative text for images
category	String (50)	nullable	PHOTO, VIDEO, DOCUMENT, OTHER
uploadedBy	String	FK -> User.id	Uploader
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@index([tenantId, category])

5.11 Library Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Book identifier

Column	Type	Constraints	Description
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
title	String (300)	NOT NULL	Book title
titleAr	String (300)	nullable	Title in Arabic
author	String (200)	nullable	Author name
isbn	String (20)	nullable	ISBN number
language	BookLanguage	DEFAULT BENGALI	Book language
category	String (100)	nullable	QURAN, HADITH, FIQH, SEERAH, GENERAL, etc.
publisher	String (200)	nullable	Publisher name
totalCopies	Int	NOT NULL	Total copies owned
availableCopies	Int	NOT NULL	Copies currently available
shelfLocation	String (50)	nullable	Physical shelf location
description	String	nullable	Book description
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, category]), @@index([tenantId, language])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Issue identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
bookId	String	FK -> Book.id	Book reference
memberId	String	FK -> User.id	Member (student/staff) reference
issueDate	DateTime	NOT NULL	Date issued
dueDate	DateTime	NOT NULL	Return due date
returnDate	DateTime	nullable	Actual return date
status	IssueStatus	DEFAULT ISSUED	Issue status
fineAmount	Decimal (8,2)	DEFAULT 0	Overdue fine amount
finePaid	Boolean	DEFAULT false	Fine payment status
issuedBy	String	FK -> User.id	Librarian who issued

Column	Type	Constraints	Description
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@index([tenantId, memberId, status]), @@index([tenantId, dueDate, status])

5.12 Hostel Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Building identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Building name
type	String (20)	DEFAULT 'BOYS'	BOYS, GIRLS, MIXED
totalRooms	Int	NOT NULL	Number of rooms
totalBeds	Int	NOT NULL	Total bed capacity
occupiedBeds	Int	DEFAULT 0	Currently occupied beds
warden	String (200)	nullable	Warden name
wardenPhone	String (20)	nullable	Warden contact
address	String	nullable	Building address
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([tenantId, name])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Room identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
buildingId	String	FK -> HostelBuilding.id, CASCADE	Building reference
roomNo	String (20)	NOT NULL	Room number
capacity	Int	NOT NULL	Bed capacity
occupied	Int	DEFAULT 0	Currently occupied

Column	Type	Constraints	Description
roomType	String (20)	DEFAULT 'SHARED'	SHARED, SINGLE, DELUXE
monthlyRent	Decimal (8,2)	nullable	Monthly rent
amenities	Json	DEFAULT '[]'	List of amenities
isActive	Boolean	DEFAULT true	Active flag

Indexes: @@unique([buildingId, roomNo])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Allocation identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
roomId	String	FK -> HostelRoom.id	Room reference
bedNo	String (10)	nullable	Bed number within room
checkInDate	DateTime	NOT NULL	Check-in date
checkOutDate	DateTime	nullable	Check-out date
status	String	DEFAULT 'ACTIVE'	ACTIVE, CHECKED_OUT, TRANSFERRED
monthlyFee	Decimal (8,2)	nullable	Monthly hostel fee
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Indexes: @@unique([studentId, status]), @@index([roomId, status])

5.13 Transport Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Vehicle identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Vehicle name/number
type	String (30)	NOT NULL	BUS, VAN, AUTO_RICKSHAW, OTHER
registrationNo	String (30)	nullable	Registration number

Column	Type	Constraints	Description
capacity	Int	NOT NULL	Passenger capacity
driverName	String (200)	nullable	Driver name
driverPhone	String (20)	nullable	Driver contact
routeId	String	FK -> TransportRoute.id, nullable	Assigned route
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Route identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
name	String (100)	NOT NULL	Route name
description	String	nullable	Route description
distance	Decimal (6,2)	nullable	Total distance (km)
estimatedTime	Int	nullable	Estimated travel time (minutes)
monthlyFee	Decimal (8,2)	nullable	Monthly transport fee
isActive	Boolean	DEFAULT true	Active flag
createdAt	DateTime	auto now()	Record creation timestamp
updatedAt	DateTime	auto updatedAt	Last modification timestamp

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Stop identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
routeId	String	FK -> TransportRoute.id, CASCADE	Route reference
name	String (200)	NOT NULL	Stop name
latitude	Decimal (10,8)	nullable	GPS latitude
longitude	Decimal (11,8)	nullable	GPS longitude
stopOrder	Int	NOT NULL	Sequence order on route

Column	Type	Constraints	Description
estimatedPickupTime	String (10)	nullable	Pickup time (HH:MM)
estimatedDropoffTime	String (10)	nullable	Dropoff time (HH:MM)

Indexes: @@unique([routeId, stopOrder])

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Rider identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution
studentId	String	FK -> Student.id, CASCADE	Student reference
routeId	String	FK -> TransportRoute.id	Route reference
stopId	String	FK -> TransportStop.id, nullable	Pickup/dropoff stop
vehicleId	String	FK -> Vehicle.id, nullable	Assigned vehicle
status	String	DEFAULT 'ACTIVE'	ACTIVE, INACTIVE
startDate	DateTime	NOT NULL	Start date
endDate	DateTime	nullable	End date
createdAt	DateTime	auto now()	Record creation timestamp

Indexes: @@unique([studentId, status]), @@index([routeId, status])

5.14 System & Configuration Tables

Column	Type	Constraints	Description
id	String (cuid)	PK, auto	Backup record identifier
tenantId	String	FK -> Tenant.id, CASCADE	Owning institution (null = full backup)
type	String (20)	NOT NULL	FULL, TENANT_DATA, FILES
filePath	String (500)	NOT NULL	Backup file path
fileSize	BigInt	NOT NULL	Backup size in bytes
status	String	DEFAULT 'COMPLETED'	PENDING, COMPLETED, FAILED
triggeredManually	Boolean	DEFAULT false	Manual trigger flag
restoredAt	DateTime	nullable	Last restore date (if any)
createdAt	DateTime	auto now()	Backup timestamp

Indexes: @@index([tenantId, createdAt])

Column	Type	Constraints	Description
key	String (100)	PK	Configuration key
value	Text	NOT NULL	Configuration value
description	String	nullable	Key description
isSecret	Boolean	DEFAULT false	Sensitive value flag (encrypted)
updatedAt	DateTime	auto updatedAt	Last update timestamp

5.15 Complete Table Inventory & ER Summary

The database schema contains 47 tables organized across the following domains. Each table includes tenant_id for multi-tenancy, standard audit timestamps, and appropriate indexes for query performance:

Domain	Tables	Count
Core Infrastructure	Tenant, Subscription, User, Role, UserRole, AuditLog	6
Student & Academic	Student, Guardian, Class, Subject, ClassSubject, Staff, Leave	7
Hifz Tracking	HifzProgress, HifzRevisionSchedule, TajweedAssessment	3
Muhasaba (Ibadah)	MuhasabaEntry, MuhasabaStreak	2
Finance & Wallet	FeeStructure, FeeAssignment, FeePayment, Expense, StudentWallet, WalletTransaction	6
Attendance	Attendance	1
Exam & Results	Exam, ExamRoutine, ExamMark	3
Communication	SMSConfig, SMSTemplate, SMSLog, Notification	4
Website / CMS	CmsPage, Notice, Event, Media	4
Library	Book, BookIssue	2
Hostel	HostelBuilding, HostelRoom, HostelAllocation	3
Transport	Vehicle, TransportRoute, TransportStop, TransportRider	4
System	BackupRecord, SystemConfig	2

Total Tables: 47 | Total Enums: 23 | Total Indexed Columns: 60+

Key Entity Relationships:

- Tenant (1:N) --> User, Student, Class, Subject, Exam, Role, FeeStructure, SMSConfig
- Student (1:N) --> Guardian, HifzProgress, MuhasabaEntry, Attendance, FeeAssignment, ExamMark
- Student (1:1) --> StudentWallet --> (1:N) WalletTransaction
- HifzProgress (1:N) --> HifzRevisionSchedule (auto-generated from spaced repetition algorithm)
- MuhasabaEntry (computed) --> MuhasabaStreak (materialized streak data)
- Class (N:M) --> Subject via ClassSubject (with teacher assignment)
- User (N:M) --> Role via UserRole (with audit trail)

5.16 Prisma Schema Examples (Critical Models)

Below are selected Prisma model definitions demonstrating the multi-tenancy pattern, enum usage, relation definitions, index declarations, and audit column conventions applied consistently across all 47 models:

```
enum HifzTier { SABAK SABAQ PARA DHOR }
enum HifzStatus { NEW MEMORIZING MEMORIZED REVISION_NEEDED COMPLETED STAGNANT }
enum TajweedGrade { EXCELLENT GOOD SATISFACTORY NEEDS_IMPROVEMENT }
enum MadrasaStream { QAWMI ALIA HIFZ NIZAMI GENERAL }
enum WalletTransactionType { ZAKAT_CREDIT ZAKAT_DEBIT SADAQAH_CREDIT SADAQAH_DEBIT SCHOLARSHIP_CREDIT EMERGENCY_DEBIT }
enum IbadahType { FAJR DHUHR ASR MAGHRIB ISHA SUNNAH QURAN_RECITATION DHIKR FASTING SADAQAH }
enum SalahQuality { ON_TIME LATE MISSED WITH_JAMAH ALONE QADA }
model HifzProgress {
  id String @id @default(cuid())
  tenantId String @map("tenant_id")
  tenant Tenant @relation(fields: [tenantId], references: [id], onDelete: Cascade)
  studentId String @map("student_id")
  student Student @relation(fields: [studentId], references: [id], onDelete: Cascade)
  juzNumber Int @map("juz_number")
  surahNumber Int @map("surah_number")
  surahName String? @map("surah_name")
  ayahFrom Int @map("ayah_from")
  ayahTo Int @map("ayah_to")
}
```

```

tier            HifzTier
status          HifzStatus @default(NEW)
memorizedDate   DateTime? @map("memorized_date")
lastRevisionDate DateTime? @map("last_revision_date")
revisionCount   Int         @default(0) @map("revision_count")
nextRevisionDate DateTime? @map("next_revision_date")
strengthScore   Int?
teacherId       String?    @map("teacher_id")
teacher         Staff?     @relation(fields: [teacherId], references: [id])
teacherRemarks String?    @map("teacher_remarks")
audioRecording  String?    @map("audio_recording")
totalAyahs      Int        @map("total_ayahs")
createdAt       DateTime   @map("created_at") @default(now())
updatedAt       DateTime   @map("updated_at") @updatedAt
@@unique([tenantId, studentId, juzNumber, surahNumber, ayahFrom])
@@index([tenantId, studentId, tier])
@@index([tenantId, studentId, status])
@@map("hifz_progress")
}

model MuhasabaEntry {
  id          String      @id @default(cuid())
  tenantId    String      @map("tenant_id")
  tenant      Tenant      @relation(fields: [tenantId], references: [id],
onDelete: Cascade)
  studentId   String      @map("student_id")
  student     Student      @relation(fields: [studentId], references: [id],
onDelete: Cascade)
  date        DateTime
  ibadahType  IbadahType  @map("ibadah_type")
  quality     SalahQuality?
  pagesRead   Int?        @map("pages_read")
  surahName   String?     @map("surah_name")
  dhikrCount  Int?        @map("dhikr_count")
  isFasting   Boolean      @default(false) @map("is_fasting")
  sadaqahGiven Boolean      @default(false) @map("sadaqah_given")
  notes       String?
  submittedBy String?     @map("submitted_by")
  createdAt   DateTime     @map("created_at") @default(now())
  updatedAt   DateTime     @map("updated_at") @updatedAt
  @@unique([tenantId, studentId, date, ibadahType])
  @@index([tenantId, studentId, date])
  @@map("muhasaba_entries")
}

model StudentWallet {
  id          String      @id @default(cuid())

```

```

    tenantId          String  @map("tenant_id")
    tenant            Tenant  @relation(fields: [tenantId], references:
[id], onDelete: Cascade)
    studentId         String  @map("student_id") @unique
    student           Student @relation(fields: [studentId], references:
[id], onDelete: Cascade)
    balance           Decimal @default(0) @db.Decimal(12, 2)
    totalCredits       Decimal @default(0) @map("total_credits")
@db.Decimal(12, 2)
    totalDebits       Decimal @default(0) @map("total_debits")
@db.Decimal(12, 2)
    isZakatEligible   Boolean @default(false) @map("is_zakat_eligible")
    needScore         Int?    @map("need_score")
    needAssessmentDate DateTime? @map("need_assessment_date")
    needAssessmentNotes String? @map("need_assessment_notes")
    priorityLevel     String  @default("NORMAL") @map("priority_level")
    isActive          Boolean @default(true) @map("is_active")
    transactions       WalletTransaction[]
    createdAt         DateTime @map("created_at") @default(now())
    updatedAt         DateTime @map("updated_at") @updatedAt
    @@index([tenantId, isZakatEligible, needScore(sort: Desc)])
    @@map("student_wallets")
}

```

Note: The complete schema.prisma file will contain all 47 model definitions. The examples above demonstrate the conventions applied consistently: tenant_id on every table, CASCADE deletes, snake_case column mapping, compound unique constraints, and strategic indexes on query patterns.

6. Project Folder Structure

The project follows Next.js 15 App Router conventions with a modular folder structure supporting scalability. Each major feature area is organized as a self-contained module with its own components, API routes, types, and utility functions.

The folder structure uses Next.js route groups (parentheses) to organize pages without affecting URL paths. The (auth) group contains authentication pages, (dashboard) contains the main application, and (public) handles tenant-specific public websites. Notable additions from v1 include the hifz/ and muhasaba/ component directories, reflecting the specialized Madrasa modules.

7. Print, PDF & Export Features

The MMS platform provides comprehensive document generation and export capabilities supporting the offline documentation needs of madrasas. These features ensure all data can be output in professional formats for printing, distribution, and archival.

- **PDF Generation:** Server-side PDF generation using Puppeteer or @react-pdf/renderer. Custom templates with institution branding. Available for report cards, fee receipts, admission forms, attendance reports, Hifz progress certificates, Muhasaba reports, Zakat disbursement reports, financial statements, and student ID cards.
- **Excel Export:** Data export to .xlsx format using ExcelJS. Available for student lists, attendance data, fee records, exam results, tabulation sheets, Hifz progress summaries, financial reports, and library inventories. Multi-sheet workbooks with formatted headers.
- **Print-Optimized Views:** Every report includes print-optimized CSS with proper page breaks, margins, and element hiding. One-click print button triggers browser print dialog with pre-configured settings.
- **Bulk Operations:** Batch PDF generation for report cards (entire class), batch ID card printing, bulk certificate generation, and bulk Hifz progress reports. Background job processing for large batches.
- **Template Customization:** Visual editor for customizing PDF templates with logo, header text, footer text, and color scheme. Template customization through institution settings panel.

8. Auto Backup System

Data protection is critical for any management system. The MMS platform implements comprehensive automated backup ensuring institutional data is regularly backed up, easily restorable, and stored securely at both platform and institution levels.

- **Scheduled Backups:** Configurable schedules (daily, weekly, monthly) with cron-based automatic execution. Reliable regardless of user activity.
- **Full Database Backup:** Complete PostgreSQL pg_dump for each institution. Includes all tables, indexes, and sequences within tenant scope.
- **File Backup:** Uploaded files and documents backup with incremental support to minimize storage and time.

- **Backup Storage:** Local storage with optional S3-compatible cloud storage. Each backup compressed and tagged with timestamp, tenant ID, and type.
- **Retention Policy:** Configurable retention (e.g., 30 daily, 12 monthly backups). Automatic cleanup of expired backups.
- **Restore Process:** One-click restore from any snapshot. Validates integrity before applying, maintains restore operation records.
- **Monitoring:** Dashboard showing backup status, last backup time, size, and failure alerts. Email notification on backup failures.

9. Security & Performance

9.1 Security Measures

Security is paramount for a multi-tenant SaaS platform handling sensitive student and financial data. The following measures are implemented across all layers:

- **Authentication:** NextAuth.js v5 with bcrypt hashing, JWT sessions, configurable expiration. Optional 2FA via TOTP.
- **Authorization:** Dynamic RBAC enforced at API route level through middleware. Every request validates permissions against required module/action.
- **Data Isolation:** PostgreSQL RLS policies as secondary defense, complementing application-level tenant_id filtering. Defense in depth.
- **Input Validation:** Zod schemas for all inputs. Prisma parameterized queries (no SQL injection). Content sanitization (XSS). CSRF tokens via NextAuth.
- **Rate Limiting:** Per-tenant and per-IP rate limiting to prevent abuse and ensure fair resource usage.
- **Encryption:** TLS/SSL for transit. Optional column encryption for sensitive fields (phone, address) using PostgreSQL pgcrypto.
- **Audit Logging:** Comprehensive logging of all data modifications (create, update, delete) with user ID, timestamp, and change details.
- **Wallet Security:** Student wallet transactions are immutable (no UPDATE/DELETE once created). All wallet modifications go through a service layer with balance validation and audit trail.

9.2 Performance Optimization

The platform handles hundreds of concurrent users across multiple tenants with responsive performance:

- Database: Prisma query optimization, 60+ strategic indexes, PgBouncer connection pooling, Redis query caching.
- Frontend: Next.js SSR, static generation for public pages, next/image optimization, code splitting, lazy loading.
- Caching: Redis for permissions, settings, translations, prayer times, streak data. HTTP cache headers for static assets.
- Background Jobs: PDF generation, bulk SMS, email sending processed through job queues to prevent request blocking.
- Materialized Views: MuhasabaStreak and attendance summaries computed as materialized views, refreshed on schedule, for instant dashboard loading.

10. Deployment Strategy

10.1 Target Infrastructure

Deployment prioritizes cost efficiency with production-grade reliability. Contabo VPS provides competitive pricing, and Coolify offers self-hosted container management comparable to Heroku/Vercel without recurring SaaS costs.

Component	Tool/Service	Purpose
Server	Contabo VPS (4 vCPU, 8GB RAM)	Primary hosting server
Container Management	Coolify (self-hosted)	Docker orchestration and deployment
Database	PostgreSQL 16 (Docker)	Application database
Cache	Redis 7 (Docker)	Query cache, sessions, rate limiting
Reverse Proxy	Traefik (via Coolify)	SSL termination and routing
File Storage	Local + S3-compatible backup	Files and backup storage
SSL	Let's Encrypt (auto via Coolify)	HTTPS certificates
CI/CD	GitHub Actions + Coolify auto-deploy	Continuous deployment

10.2 Docker Setup

The entire application is containerized using Docker for consistent deployment. The docker-compose.yml defines four primary services: Next.js application, PostgreSQL database, Redis cache, and optional MinIO for S3-compatible storage. Each service runs in its own container with resource limits configured to prevent resource monopolization. The Dockerfile uses multi-stage builds to minimize final image size.

10.3 Scaling Strategy

Initial deployment targets a single VPS suitable for early customers. As the base grows, horizontal scaling adds more VPS instances behind a load balancer. The stateless Next.js container allows easy horizontal scaling. PostgreSQL can upgrade to a managed service for better performance. Redis clustering provides cache scalability.

11. Development Roadmap

The development follows a phased approach delivering a working product incrementally. Each phase builds on the previous, allowing early testing and feedback. The roadmap prioritizes core functionality first, then expands to advanced features:

Phase	Duration	Key Deliverables
Phase 1: Foundation	4-5 weeks	Project setup, auth, RBAC, multi-tenancy, institution settings, user management
Phase 2: Core Modules	6-7 weeks	Student management, teacher management, attendance, class/subject management
Phase 3: Hifz & Academic	5-6 weeks	Hifz 3-tier tracking, Tajweed assessment, exam management, report cards, tabulation
Phase 4: Muhasaba & Spiritual	3-4 weeks	Muhasaba module, streak tracking, prayer times, Salah tracking, gamification
Phase 5: Finance & Wallet	4-5 weeks	Fee management, student wallet, Zakat/Sadaqah tamlik, payment gateways, expenses
Phase 6: Communication	2-3 weeks	SMS system, notification center, notice/event management, bulk operations

Phase	Duration	Key Deliverables
Phase 7: Website/CMS	3-4 weeks	Public website builder, templates, notice board, event calendar, media gallery
Phase 8: Secondary Modules	3-4 weeks	Library, hostel, transport modules (optional per institution plan)
Phase 9: Polish & Backup	3-4 weeks	Backup system, print/PDF optimization, UI refinement, performance tuning
Phase 10: Testing & Launch	2-3 weeks	Comprehensive testing, UAT, security audit, load testing, beta launch

Total estimated timeline: approximately 35-45 weeks (9-11 months) for production-ready launch, assuming 2-3 full-time developers. Phases 3-6 include parallel work streams (e.g., Hifz and Exams can overlap, Finance and Communication can be simultaneous).

12. Risk Analysis & Mitigation

Risk	Impact	Probability	Mitigation Strategy
Scope creep from custom requests	High	High	Strict module boundaries, feature freeze per phase, configurable settings
Performance with many tenants	Medium	Medium	60+ strategic indexes, PgBouncer, Redis caching, horizontal scaling plan
SMS delivery failures	Medium	Medium	Configurable provider fallback, retry mechanism, delivery tracking
Data loss from backup failure	High	Low	Multiple backup locations, integrity checks, regular restore testing
Security breach	Critical	Low	RBAC + RLS, input validation, rate limiting, security audit before launch

Risk	Impact	Probability	Mitigation Strategy
Low adoption by madrasa admins	High	Medium	Intuitive UI, Bangla-first design, guided onboarding, video tutorials
Payment gateway issues	Medium	Medium	Abstraction layer for providers, thorough testing, manual fallback
Zakat wallet compliance concerns	High	Low	Immutable transactions, full audit trail, Shariah compliance review

13. Additional Features & Competitive Advantages

- Beyond the core modules described above, the following features provide competitive advantage and ensure completeness:
- **Dashboard Analytics:** Institution-specific dashboard with KPIs, trends, and charts. Module-specific dashboards for every feature area.
 - **Prayer Time Integration:** Automatic prayer time display based on institution location. Integrated with Muhasaba for Salah tracking. Next prayer countdown widget.
 - **Notice Board:** Create and publish notices with priority levels, categories, and expiration dates. Visible in dashboard and optionally on public website.
 - **Event Calendar:** Manage institutional events with date, time, venue. Display on dashboard and public website. Islamic calendar integration.
 - **Mobile Responsive:** Fully responsive design usable on mobile and tablets. Critical for teachers marking attendance on phones.
 - **Dark Mode:** System-wide dark theme with CSS custom properties. Toggle in settings. Respects prefers-color-scheme.
 - **Activity Logging:** Comprehensive log showing all user actions with filter by user, action type, and date range.
 - **Data Import/Export:** CSV/Excel import for students, staff, fees. Bulk operations for efficiency.
 - **Onboarding Wizard:** Step-by-step guided setup for new institutions ensuring all essential configurations are completed.

- **Bangla-First Design:** Entire interface designed with Bangla as primary language. Proper typography, number formatting (Bengali numerals option), and cultural conventions.
- **Islamic Calendar:** Hijri date display alongside Gregorian. Islamic event awareness (Ramadan, Eid, Shab-e-Qadr).
- **QR Code Student IDs:** Auto-generated QR code on student ID cards for quick attendance scanning via mobile.